Yoneda products for the C-motivic minimal resolution

An implementation note for `yoneda.cpp`

June 23, 2026

Abstract

This note explains the mathematics behind `yoneda.cpp`, a tool that computes Yoneda (composition) products on the Ext groups produced by Guozhen Wang's `MinimalResolution` infrastructure for the C-motivic Steenrod algebra. The existing code already computed multiplication by the filtration-1 classes h_n (this is `mot_mult`); we generalize this to multiplication by an *arbitrary* class via chain-map lifting. I assume comfort with homological algebra (resolutions, Ext, Yoneda products) but treat the motivic apparatus as a black box, calling out only the two features that actually change the algorithm: the ground ring is $\mathbb{F}_2[\tau]$ rather than a field, and the relevant modules can have τ-torsion.

1 The homological setup, with the motivic part boxed off

What is being resolved. Fix the ground ring $\Bbbk = \mathbb{F}_2[\tau]$, a graded polynomial ring on one generator τ. (Black box: $\Bbbk$ is the mod-2 motivic cohomology of a point; τ is a non-nilpotent element of homological degree 0. Setting $\tau = 1$ recovers the classical mod-2 story, and "τ-torsion" is the genuinely new phenomenon.) There is a graded Hopf algebroid $(\Bbbk, \mathcal{A})$ – the C-motivic dual Steenrod algebra – and we work in the category of $\mathcal{A}$-comodules. The object of interest is

$$\mathrm{Ext}_{\mathcal{A}}^{s,*}(\Bbbk, \Bbbk),$$

the E_2-page of the C-motivic Adams spectral sequence for the sphere. Each group is a graded $\Bbbk = \mathbb{F}_2[\tau]$-module.

Cofree comodules and the resolution. For a graded $\Bbbk$-module V there is a *cofree* (relatively injective) comodule $\mathcal{F}(V) = \mathcal{A} \,\square\, V$, characterized by the adjunction

$$\mathrm{Hom}_{\mathcal{A}}\big(M, \mathcal{F}(V)\big) \;\cong\; \mathrm{Hom}_{\Bbbk}(M, V), \tag{1}$$

natural in the comodule M: a comodule map into $\mathcal{F}(V)$ is the same thing as a $\Bbbk$-linear map into the "cogenerators" V. Explicitly, a $\Bbbk$-linear $\psi\colon M \to V$ corresponds to the comodule map

$$\widehat{\psi}(m) = (\mathrm{id}_{\mathcal{A}} \otimes \psi)\,\big(\psi_M(m)\big) \in \mathcal{A}\square V = \mathcal{F}(V), \tag{2}$$

where $\psi_M\colon M \to \mathcal{A} \otimes M$ is the coaction. Formula (**??**) is the computational workhorse below.

`MinimalResolution` builds a *minimal* resolution of the trivial comodule $\Bbbk$ by cofree comodules,

$$0 \to \Bbbk \to F_0 \xrightarrow{d_0} F_1 \xrightarrow{d_1} F_2 \to \cdots, \qquad F_s = \mathcal{F}(V_s), \tag{3}$$

i.e. an injective resolution in comodules. Write G_s for the cogenerators of F_s (a finite free $\Bbbk$-module; in the code these are `F.generators`). Because the resolution is minimal, the induced differential on cogenerators vanishes *modulo* τ, and one has, up to the τ-Bockstein bookkeeping discussed in §**??**,

$$\mathrm{Ext}^{s}_{\mathcal{A}}(\Bbbk,\Bbbk) \;\cong\; G_s \qquad \text{(a basis is the cogenerators of } F_s\text{)}.$$

A class is therefore named by a pair $\{s\text{-}a\}$: homological filtration s and the index a of a cogenerator of F_s. For example $h_n = \{1\text{-}n\}$.

How the resolution is stored. Each step is realized by a short exact sequence of comodules

$$0 \to X_s \xrightarrow{\iota_s} F_s \xrightarrow{q_s} X_{s+1} \to 0, \qquad X_0 = \Bbbk, \tag{4}$$

with the resolution differential $d_s = \iota_{s+1} \circ q_s \colon F_s \to F_{s+1}$. The comodules X_s are the (co)syzygies. On disk one finds, for each s: the cofree comodule F_s (file `mot_gens`s) and the two $\Bbbk$-linear maps ι_s, q_s (file `mot_maps`s), all as sparse matrices over $\mathbb{F}_2[\tau]$. Every map in sight is an honest $\mathbb{F}_2[\tau]$-linear map between finitely generated free $\mathbb{F}_2[\tau]$-modules; the comodule structure enters only through the coactions, via (**??**).

2 Yoneda products via chain-map lifting

The classical recipe. A class $\beta \in \mathrm{Ext}^{s'}(\Bbbk,\Bbbk)$ is represented by a map $\Bbbk \to \Bbbk[s']$ in the derived category, equivalently (using (**??**) as an injective resolution of the target) by a *chain map*

$$f_\bullet \colon F_\bullet \longrightarrow F_{\bullet+s'}, \qquad d_{i+s'}\, f_i = f_{i+1}\, d_i,$$

that lifts β in degree 0. The Yoneda product $\alpha \cdot \beta$, for $\alpha \in \mathrm{Ext}^{s}$, is obtained by applying f_s to a cocycle representing α and reading the result in $\mathrm{Ext}^{s+s'}$. Two chain maps lifting β differ by a homotopy, so the induced map on cohomology – the product – is well defined. This is exactly the "simplest approach": build $f_\bullet$ by lifting, one filtration at a time.

The base of the lift. $F_0 = \mathcal{F}(\Bbbk)$ is cofree on a single cogenerator u in degree 0. We set f_0 to be the comodule map sending u to the cogenerator b of $F_{s'}$ that *is* $\beta = \{s'\text{-}b\}$. Via (**??**) this is encoded by the $\Bbbk$-linear map $\psi_0 \colon F_0 \to G_{s'}$ with $\psi_0(u) = e_b$ and $\psi_0 = 0$ on the rest of a $\Bbbk$-basis of F_0; then $f_0 = \widehat{\psi_0}$ by (**??**).

The inductive step. Assume $f_i \colon F_i \to F_{i+s'}$ is built. We must produce $f_{i+1} \colon F_{i+1} \to F_{i+s'+1}$ with $f_{i+1}d_i = d_{i+s'}f_i$. The key points:

1. f_{i+1} **is forced on the image of** d_i**.** Since $d_i = \iota_{i+1}q_i$ and $\ker d_i = \ker q_i = \mathrm{im}\,\iota_i$, the chain-map equation determines f_{i+1} on $\mathrm{im}\, d_i = \mathrm{im}\,\iota_{i+1}$ by

   $$f_{i+1}\big(d_i(e)\big) = d_{i+s'}\big(f_i(e)\big), \qquad e \in F_i, \tag{5}$$

   and this is consistent precisely because β is a cocycle (the obstruction $d_{i+s'}f_i|_{\mathrm{im}\,\iota_i}$ vanishes by the previous step).

2. **Cogenerators live in that image.** The cogenerators of F_{i+1} are the leading terms of ι_{i+1}, hence lie in $\mathrm{im}\,\iota_{i+1} = \mathrm{im}\, d_i$. So (**??**) computes f_{i+1} *on cogenerators*, which by (**??**) is all we need to reconstruct f_{i+1} as a comodule map.

Concretely, to compute f_{i+1} on the cogenerator w (a basis vector of F_{i+1}):

$$u := \iota_{i+1}^{-1}(w) \in X_{i+1}, \quad e := q_i^{-1}(u) \in F_i, \quad f_{i+1}(w) := d_{i+s'}\big(f_i(e)\big). \tag{6}$$

Here ι_{i+1}^{-1} means the unique preimage under the *injection* ι_{i+1} (it exists because $w \in \operatorname{im}\iota_{i+1}$), and q_i^{-1} means *any* preimage under the surjection q_i; then $d_i(e) = \iota_{i+1}(q_i(e)) = \iota_{i+1}(u) = w$, so (**??**) is just (**??**). Finally we package $\{w \mapsto f_{i+1}(w)\}$ back into the full comodule map via the adjoint (**??**); and $f_i(e)$ for the general element e in (**??**) is itself evaluated by (**??**) applied to $f_i = \widehat{\psi_i}$.

Reading off the product. This yields, on cogenerators, a $\Bbbk$-linear "multiplication matrix" $M_s\colon G_s \to G_{s+s'}$, $M_s = \big(\text{project to cogenerators}\big) \circ f_s$. For $\alpha = \{s\text{-}a\}$,

$$\alpha \cdot \beta \;=\; M_s\big(\text{cocycle of } \alpha\big) \;\in\; \operatorname{Ext}^{s+s'},$$

re-expressed in the chosen basis of $\operatorname{Ext}^{s+s'}$ (§**??**). Specializing $\beta = h_n$ recovers exactly the construction in `mot_mult` (there the chain map is the elementary one "multiply by the algebra element h_n, then apply one differential"); our code reproduces those tables identically as a sanity check.

3 The one motivic complication: τ-torsion and τ-aware solving

Everything above is standard homological algebra. The single place where "motivic" intervenes is in computing the two preimages in (**??**), i.e. in inverting

$$\iota_{i+1} \text{ on its image} \qquad \text{and} \qquad q_i \text{ (a surjection)}.$$

Over a field this is ordinary Gaussian elimination. Over $\Bbbk = \mathbb{F}_2[\tau]$ it is not, and naive elimination fails.

Why a field-style splitting does not exist. $\mathbb{F}_2[\tau]$ is a PID, and a short exact sequence (**??**) of $\Bbbk$-modules splits iff $X_{s+1} = \operatorname{coker}\iota_s$ is free, i.e. *τ-torsion-free*. C-motivic Ext is famous for having τ-torsion, so the syzygies are in general not free, and ι_s has *no global retraction* $F_s \to X_s$. (Empirically: in the 10-cell test resolution, attempting a global τ^0-pivoted retraction already fails at step 1, on the row whose ι_1-image is τ-divisible.) This is exactly why the existing code never lifts chain maps over $\mathbb{F}_2[\tau]$ – and the conceptual obstacle the present work had to get past.

The resolution of the difficulty. We never need a global retraction. In (**??**) we only ever

- invert ι_{i+1} on an element w *known to lie in* $\operatorname{im}\iota_{i+1}$ (a cogenerator); since ι_{i+1} is injective the preimage is unique and *always exists*; and
- invert q_i on an arbitrary target element; since q_i is surjective a preimage *always exists*.

Existence is guaranteed by exactness of (**??**); there is no torsion obstruction to *solving* these particular systems. What is required is an elimination that works over the PID $\mathbb{F}_2[\tau]$.

τ-aware (Hermite) elimination. The implementation builds, for each matrix ι_i and q_i, a row-echelon decomposition over $\mathbb{F}_2[\tau]$ in which the pivot at each position is kept with the *minimal* τ-power seen so far (a one-prime Hermite normal form; the only relevant prime is τ). Reducing a target w against these pivots either certifies $w \notin$ im (a residual survives) or returns a preimage, with the elimination introducing τ^k factors ($k \geq 0$) exactly when the system forces them. This is the class `TauEchelon` in the code; the same routine serves both $\iota^{-1}|_{\text{im}}$ and the section of q.

Where the τ's in products come from. The τ-powers produced by this elimination are precisely the source of τ-multiples in Yoneda products. The cleanest example: classically $h_0^2 h_2 = h_1^3$, while our computation returns

$$h_1^3 = \{3\text{-}1\}, \qquad h_0^2 h_2 = \tau \cdot \{3\text{-}1\},$$

i.e. the C-motivic refinement $\boxed{h_0^2 h_2 = \tau\, h_1^3}$, with the τ appearing exactly as the Hermite elimination's τ^1 factor.

4 Naming the answer: the τ-Bockstein basis

One subtlety remains in how Ext is presented. Because the minimal differential is τ-divisible, the honest $\mathbb{F}_2[\tau]$-module Ext^s is not simply "the cogenerators of F_s": one must run the τ-Bockstein spectral sequence to find the correct cycle representatives and the τ-towers. This is already implemented (`make_table`/`make_cycle_tables` in `tao_bockstein.cpp`), producing for each filtration a distinguished set of cycle representatives (`cyc_table`). Our product routine names its output in this basis, via the existing `find_cycle`, so that the result is directly comparable with the h_n-multiplication tables and with the literature. The τ-exponent printed as `t^k` in front of a generator is the τ-power attached by this naming.

5 Summary of the algorithm and verification

Algorithm (file `yoneda.cpp`).

1. Load the resolution $F_\bullet$ and the maps ι_s, q_s; build τ-aware solvers $\texttt{invInj}_s$ (invert ι_s on its image) and $\texttt{secQut}_s$ (section of q_s). A self-check verifies $\iota_s^{-1}\iota_s = \mathrm{id}$ and $q_s\, q_s^{-1} = \mathrm{id}$ on the relevant spaces.
2. Build the τ-Bockstein cycle representatives (reusing `make_table`).
3. For the chosen $\beta = \{s'\text{-}b\}$, lift the chain map $f_\bullet$ by (**??**), using (**??**) to evaluate and to reassemble comodule maps. Record the cogenerator-level matrices M_s.
4. Output either a single product $\alpha \cdot \beta$, or the full table $\beta \cdot (-)$ over all generators, named in the τ-Bockstein basis.

A technical point in step 3: when a product's internal degree exceeds the resolution range, (**??**) would place terms outside the relevant cofree summand; these are degree-overflow artifacts and are clipped to the summand, which is correct within range (nothing valid is dropped) and matches `mot_mult`'s degree guard.

Verification. On a test resolution (`md=10`, length 6):

- The solver self-check passes at every filtration.
- Taking $\beta = h_n$ reproduces `mot_mult`'s tables $\{n = 0, 1, 2, 3\}$ *exactly*, including τ-multiples such as $h_0 \cdot \{2\text{-}2\} = \tau\{3\text{-}1\}$ and $h_0 h_1 = 0$.
- Genuinely higher-filtration (input) products agree with known relations: $h_1^2 = \{2\text{-}1\}$, $h_1^3 = \{3\text{-}1\}$, and $h_0^2 h_2 = \tau\{3\text{-}1\} = \tau h_1^3$.
- Single-pair queries agree with the corresponding entries of the full tables.

Usage. After the bootstrap `e2p` md; `motTab` md; `mr_ex` md L_{ex}; `mr_mot` md L (note: `motTab` is required), run

`yoneda` md L	solver self-check
`yoneda` md L b_s b_b	full table for $\beta = \{b_s\text{-}b_b\}$
`yoneda` md L a_s a_a b_s b_b	single product $\{a_s\text{-}a_a\} \cdot \{b_s\text{-}b_b\}$

where md is half the maximal topological degree and L the resolution length.

Remark 1 (scope)**.** Only Yoneda products are implemented. Massey products are a natural sequel: the τ-aware solving developed here already produces the null-homotopies of vanishing products that a Massey-product implementation would consume; what remains is the indeterminacy bookkeeping.